

fan.livelatch.com

Alpha MVP Go-Live — Implementation Guide

Version 1.0 · 2026-07-06 · docs/fanservice/alpha-mvp-golive · Author: Claude (Fable), reviewed by Alex

1. What this guide covers

The LatchDeck fan client (repo G:\repos\fan-livelatch) is complete as a mock: redeem flow, LatchID sign-in modal, first-run username claim, public cabinets with cards and trophies. Every external dependency sits behind two seams — `lib/latchdeck/client.ts` (data) and `lib/auth/session.ts` (auth) — so going live means implementing those seams and deploying, not touching the UI.

Go-live is four workstreams, in dependency order: **Encore endpoints** → **fan-app live wiring** → **platform config (Supabase/Railway/DNS)** → **verification**. The final section is the execution prompt to hand to Claude, including the decision points and the manual steps that only you can perform.

2. Target architecture

Alpha architecture — the fan app is a thin keyed client; nothing Laravel-side changes.

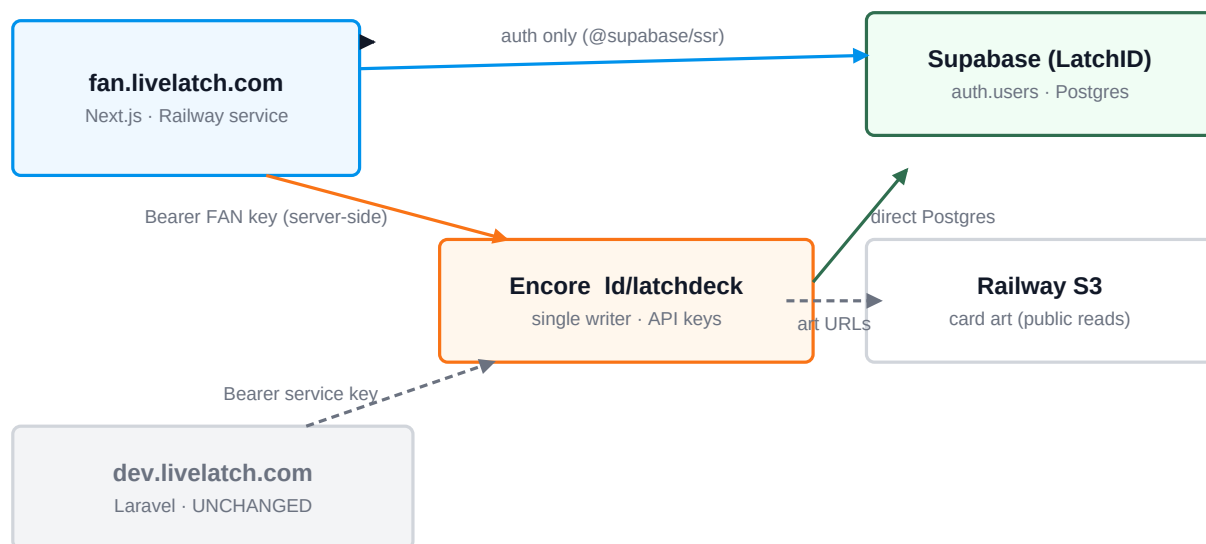


Figure 1 — Alpha architecture. The fan app talks to exactly two things: Supabase (auth only) and Encore (all LatchDeck data).

Two rules carry the whole design: **Encore is the only writer** of `latchdeck_*` tables (the fan app's Supabase client is used for authentication and nothing else), and **fans are never replicated into Laravel** — the Laravel MySQL user table remains a creators-only concern.

3. Workstream 1 — Encore (Id/latchdeck)

The bulk of the work. Everything else is configuration by comparison.

3.1 Migration (additive only)

- `latchdeck_fans` — `latchid_user_id` (PK → `auth.users`), `username` (unique, lowercase), `display_name`, `avatar_url`, `is_public` bool default true, `created_at`.
- `latchdeck_card_instances` — `id` uuid(v7), `card_id` → `latchdeck_cards_mvp`, `owner_id` → `auth.users`, `serial_no` int, `redeemed_via`, `minted_at`. Constraints: **UNIQUE (card_id, serial_no)** and **UNIQUE (card_id, owner_id)** — races physically cannot double-mint or double-own.
- `latchdeck_ownership_events` — append-only provenance ledger (`instance_id`, `from_identity`, `to_identity`, `reason` 'mint', `at`). Written from day one so trading needs no backfill.

3.2 Endpoints

- POST `/redeem` — { `latchid_user_id`, `code` } → find published card by `redeem_code_mvp`; check `is_active_mvp`, supply cap, not already owned; allocate next serial **inside one transaction**; insert instance + mint event. Distinct error codes: `invalid_code` / `closed` / `sold_out` / `already_owned` (the fan UI already renders each).
- POST `/fans` — username claim; validate against the existing `username_blacklist` and `impersonation_names` tables (reuse, don't rebuild).
- GET `/fans/by-username/:username` and GET `/fans/:latchid_user_id/collection` — what the public cabinet renders.

3.3 Auth & abuse

- New `LATCHDECK_FAN_API_KEY` in `auth.ts` — per-client keys mean the fan app can be revoked without touching Livelatch, and it is the pattern the SDK tier needs anyway.
- Rate-limit `/redeem` per user and per IP — shared codes invite brute-force guessing of other codes.

Redeem happy path — checks at step 3: `code valid` · `edition active` · `cap not exhausted` · `one-per-fan`.



Failure at any check returns a typed error (`invalid_code` / `closed` / `sold_out` / `already_owned`) — the UI already renders each.

Figure 2 — The redeem path and its four typed failure modes.

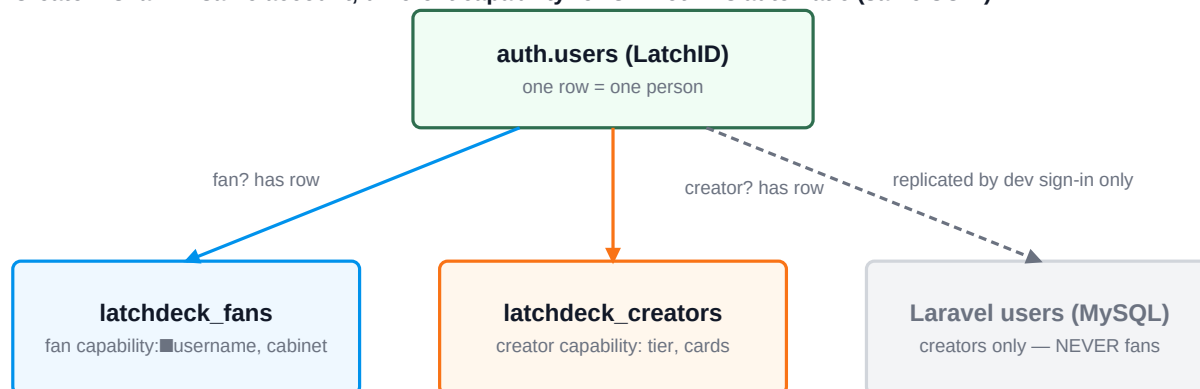
4. Workstream 2 — Fan app live wiring

- Implement `lib/latchdeck/encore.ts` — fetch calls with the Bearer fan key, server-side only (the key never reaches the browser). The expected endpoints are already documented in that file's comments.
- Implement `lib/auth/session.ts` with `@supabase/ssr` against the LatchID project: Google OAuth + email OTP, an `/auth/callback` route, sessions in cookies. The LatchID modal remains the only sign-in surface; passkeys can land post-alpha (RP ID `livelatch.com` already covers the subdomain).
- `APP_DATA_MODE=mock` stays the local default. Live mode activates only when the env vars are present — a misconfigured deploy fails safe into mock.

- PostHog (project 201558, EU host, cookieless): fan_signed_up, card_redeemed (rarity/serial props), cabinet_viewed.
- On fan signup, create a user_email_preferences row with everything off — GDPR posture matches the main app; fans get no email until they opt in.

5. Identity — creator vs fan (or both)

Creator vs fan — same account, different capability rows. “Both” is automatic (same UUID).



Do not gate on `profiles.user_type` (defaults to 'creator'); derive status from capability rows instead.

Figure 3 — One LatchID, capability rows decide what a person is. Ben is both without any linking code.

- A **fan** is anyone with a `latchdeck_fans` row (created at username claim).
- A **creator** is anyone with a Laravel user / `latchdeck_creators` row (created by the existing `dev.livelatch.com` flow).
- **Both** is the same `auth.users` UUID having both rows — automatic, no join table, no linking step.
- Do not write `profiles.user_type` from the fan flow (it defaults to 'creator' and predates fans). Derive status from capability rows — derived truth cannot drift.

6. Workstream 3 — Platform configuration

Deploy topology — new service in the existing project; Laravel and MySQL untouched.

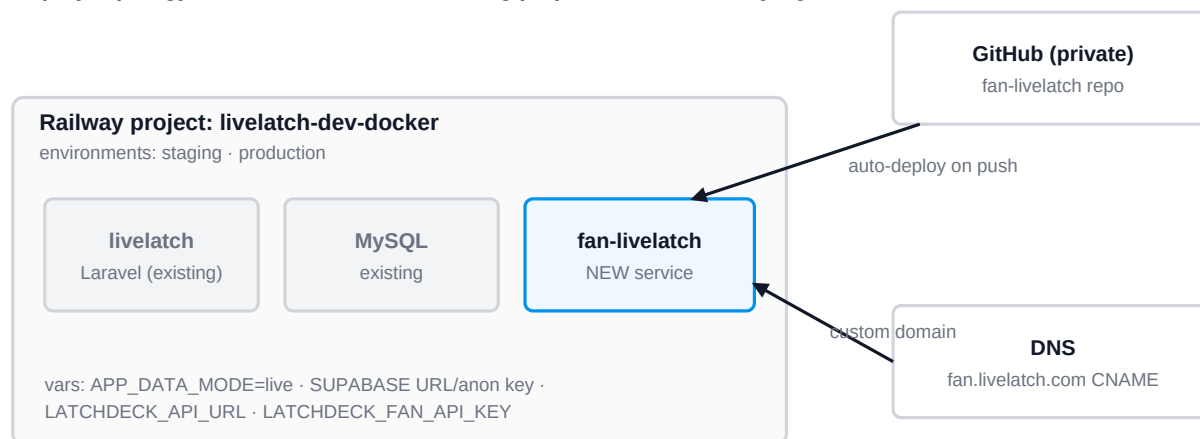


Figure 4 — Railway: one new service in the existing project. Per-service isolation is already the failure boundary.

6.1 Supabase (LatchID project `yalyfdfnphxzuhqlbfs`)

- Auth → URL Configuration → Redirect URLs: add `https://fan.livelatch.com/**` (or the `https://*.livelatch.com/**` wildcard once, covering all future services).
- Nothing changes in Google Cloud — Google's redirect URI is Supabase's own callback and is already set.
- No new providers — Google OAuth and email OTP are configured project-wide already.

6.2 Railway

- New service **fan-livelatch** in the existing `livelatch-dev-docker` project, connected to the private GitHub repo. Next.js auto-builds (`npm run build / start, $PORT` respected).
- Variables per `.env.example`: `APP_DATA_MODE=live`, `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `LATCHDECK_API_URL`, `LATCHDECK_FAN_API_KEY`, PostHog keys.
- Custom domain `fan.livelatch.com` → CNAME to the Railway service.

6.3 Encore Cloud

- Decide staging vs production environment. For a real alpha, create Encore's production env — otherwise a staging redeploy interrupts live fans mid-drop. Fan key goes into Encore secrets.

7. Hard safety rails

These are the “nothing else breaks” guarantees. They are restated inside the execution prompt.

- **No Laravel changes.** `dev.livelatch.com` auth, routes, session bridge, and Stripe flows are out of scope entirely.
- **Additive-only database changes**, applied via Encore migrations. No ALTERs to existing `latchdeck_*` tables, no RLS changes to existing tables.
- **Single writer holds.** Neither the fan app nor Laravel ever writes `latchdeck_*` tables directly.
- **No fan replication.** Zero new rows in Railway MySQL users from any fan flow — verified explicitly before sign-off.
- **Fail-safe deploys.** Missing env vars fall back to mock mode, never to a half-configured live mode.
- **Stop-points.** Migration SQL against the live LatchID project is shown and approved before it runs; the same applies to anything irreversible.

8. Verification checklist (definition of done)

- Deployed URL, fresh test account: code → LatchID modal sign-in → username claim → mint with correct next serial → public cabinet renders.
- All four redeem error states verified live.
- Laravel MySQL users row count unchanged by the entire fan flow.
- PostHog events arriving on project 201558.
- Supabase advisors show nothing new (`get_advisors`).
- Serial integrity: two concurrent redeems of the same card never produce duplicate serials (constraint holds under load).

9. The execution prompt

Paste the block below into a Claude Code session in `G:\repos\livelatch-dev-docker` when you are ready to execute. It encodes the decisions above as defaults, forces phase-by-phase check-ins, and lists the manual steps Claude must hand back to you. A copy lives alongside this PDF as `prompt.md` for easy copying.

```
Take fan.livelatch.com from mock to alpha, following the guide in
docs/fanservice/alpha-mvp-golive (read the PDF first). Work phase by
phase; at the end of each phase show me what changed and WAIT for my
go-ahead before starting the next phase.

MY DECISIONS (treat as fixed unless you flag a real blocker):
1. Auth: per-app callback at fan.livelatch.com/auth/callback behind the
existing LatchID modal. Do NOT modify dev.livelatch.com's
/callback/google or the Laravel session bridge in this task.
2. Railway: new service "fan-livelatch" inside the existing
livelatch-dev-docker project. Do not touch the Laravel or MySQL
services.
3. Encore: dedicated LATCHDECK_FAN_API_KEY (never reuse the Livelatch
service key). Target staging first; we promote to a prod env after
verification.
4. Identity: derive fan/creator status from capability rows
(latchdeck_fans / latchdeck_creators). Never write
profiles.user_type from the fan flow. Fans must never create rows
in Laravel MySQL.

PHASE 1 - Encore (G:\repos\ld\latchdeck):
- Additive migration: latchdeck_fans; latchdeck_card_instances with
UNIQUE (card_id, serial_no) and UNIQUE (card_id, owner_id);
latchdeck_ownership_events (append-only).
- Endpoints: POST /redeem (transactional serial allocation, typed
errors: invalid_code/closed/sold_out/already_owned), POST /fans
(username claim, checked against username_blacklist and
impersonation_names), GET /fans/by-username/:username,
GET /fans/:latchid_user_id/collection.
- Add LATCHDECK_FAN_API_KEY handling to auth.ts. Rate-limit /redeem
per user and per IP.
- CHECK: show me the full migration SQL and STOP for my explicit OK
before applying anything to the live LatchID Supabase project.

PHASE 2 - Fan app live wiring (G:\repos\fan-livelatch):
- Implement lib/latchdeck/encore.ts against the Phase 1 endpoints
(server-side Bearer key only; never NEXT_PUBLIC).
- Implement lib/auth/session.ts with @supabase/ssr (Google + email
OTP) and an /auth/callback route; the LatchID modal stays the only
sign-in UI. Passkeys are post-alpha.
- Keep APP_DATA_MODE=mock as the default; live mode only via env.
- PostHog project 201558 (EU, cookieless): fan_signed_up,
card_redeemed, cabinet_viewed. Create user_email_preferences rows
(all off) on fan signup.
- CHECK: demo the full flow to me locally against Encore staging
before we deploy anywhere.

PHASE 3 - Deploy. These are MANUAL steps that are mine; ask me to
confirm each one as you reach it, do not attempt them yourself:
[ ] I have created the private GitHub repo and pushed fan-livelatch
(I do my own commits/pushes via GitHub Desktop).
[ ] I have connected the repo as a new Railway service and you may
set its variables.
[ ] I have added https://fan.livelatch.com/** to the Supabase Auth
redirect allowlist (ask me whether I want the *.livelatch.com
```

wildcard instead - my call).

[] I have set LATCHDECK_FAN_API_KEY in Encore Cloud secrets.

[] I have added the DNS CNAME for fan.livelatch.com.

PHASE 4 - Verify before you call it done:

- Fresh test account on the deployed URL: code -> sign-in -> username claim -> mint with correct serial -> public cabinet.
- All four redeem error states, live.
- Laravel MySQL users table row count unchanged by the fan flow.
- PostHog events arriving; supabase get_advisors reports nothing new.

HARD SAFETY RAILS (non-negotiable):

- Encore stays the single writer of latchdeck_* tables.
- Additive-only DB changes; no destructive migrations; no RLS changes to existing tables; no edits to existing latchdeck_* columns.
- Zero changes under dev.livelatch.com (auth, routes, Stripe).
- If anything fails against the live Supabase project, stop and report - do not retry with variations.
- Log the work in summary.md per repo convention when finished.